

Design Concepts in Databases and Software Engineering

Understanding these concepts helps in designing scalable, maintainable, and efficient systems. Below is a breakdown of key principles and techniques.

1. CAP Theorem (Consistency, Availability, Partition Tolerance)

CAP theorem states that in a distributed system, you can only achieve two out of the three guarantees:

Property	Description
Consistency (C)	Every read receives the latest write (strong consistency).
Availability (A)	The system always responds, even if some nodes fail.
Partition Tolerance (P)	The system continues to function despite network partitions.

- ◆ **Trade-offs:**
 - **CP (Consistency + Partition Tolerance):** Sacrifices availability (e.g., MongoDB with strong consistency settings).
 - **AP (Availability + Partition Tolerance):** Sacrifices consistency (e.g., DynamoDB, Cassandra).
 - **CA (Consistency + Availability):** Not achievable in distributed systems.
-

2. SOLID Principles (Object-Oriented Design)

These five principles help create maintainable and scalable software.

Principle	Description
S - Single Responsibility	A class should have only one reason to change.
O - Open/Closed	Code should be open for extension but closed for modification.
L - Liskov Substitution	Subtypes should be replaceable for their base types without altering correctness.
I - Interface Segregation	Avoid large interfaces; use multiple specific ones.
D - Dependency Inversion	Depend on abstractions, not concretions.

♦ Example:

Bad:

```
class Report {  
    void generatePDF() { /* logic */ }  
    void generateExcel() { /* logic */ }  
}
```

Good (Single Responsibility Principle applied):

```
interface ReportGenerator { void generate(); }  
class PDFReport implements ReportGenerator { public void generate() { /* PDF logic */ }  
class ExcelReport implements ReportGenerator { public void generate() { /* Excel logic */ } }
```

3. ACID Principles (Database Transactions)

Ensures reliable database transactions.

Property	Description
Atomicity (A)	Transactions are all-or-nothing.
Consistency (C)	Transactions bring the database from one valid state to another.
Isolation (I)	Concurrent transactions execute independently.
Durability (D)	Once committed, a transaction remains even after system failure.

♦ **Example:** Banking transaction

1. Debit ₹500 from Account A
 2. Credit ₹500 to Account B
 - ✓ If step 2 fails, step 1 should be rolled back (Atomicity).
-

4. Database Normalization Forms

Reduces redundancy and improves integrity.

Normal Form	Rule
1NF (First Normal Form)	No duplicate rows; each column holds atomic values.
2NF (Second Normal Form)	1NF + no partial dependencies on primary key.
3NF (Third Normal Form)	2NF + no transitive dependencies.
BCNF (Boyce-Codd NF)	3NF + more strict conditions on functional dependencies.

♦ Example:

✗ Non-1NF (Repeating groups):

OrderID	Product	Product2
101	Laptop	Mouse

✓ 1NF Corrected:

OrderID	Product
101	Laptop
101	Mouse

5. Sharding vs. Fragmentation (Data Partitioning)

Concept	Description
Sharding	Splitting a database into smaller, faster, more easily managed parts (shards).
Fragmentation	Splitting data logically based on attributes, either horizontally (rows) or vertically (columns).

♦ Example of Sharding:

- A large customer database is sharded based on **region (USA, EU, Asia)** to improve performance.

♦ Example of Fragmentation:

- **Horizontal Fragmentation:** Split a student table into **UG students** and **PG students**.
 - **Vertical Fragmentation:** Store **personal details** in one table and **academic details** in another.
-

6. Optimizing Search with Indexing

♦ Types of Indexes:

- ✓ **B-Tree Index** (Default for most databases, balances tree structure).
- ✓ **Hash Index** (Fast lookups but inefficient for range queries).
- ✓ **Full-Text Index** (Used for text searching).

♦ Optimization Techniques:

1. **Use Composite Indexes** for multiple columns.

```
CREATE INDEX idx_user ON users (last_name, first_name);
```

2. **Covering Index**: Ensures all queried columns are part of the index.
3. **Use Partitioning**: Split large tables to optimize search.
4. ****Avoid SELECT *****: Fetch only required columns to reduce I/O.

Conclusion

- Use **CAP theorem** to design distributed systems.
- Follow **SOLID principles** for maintainable software.
- Ensure **ACID compliance** in critical transactions.
- Apply **Normalization** for efficient storage.
- Use **Sharding/Fragmentation** for scalability.
- Optimize **search with indexes** for faster queries.